ECR

16/4/26 22:13

Amazon Elastic Container Registry
Cheat Sheet

- A managed AWS Docker registry service that operates at the **regional level**. This design provides flexibility in how images are deployed. You can push and pull images to and from the same AWS Region where your Docker cluster is running, ensuring optimal performance.
- **Amazon Elastic Container Registry (Amazon ECR)** is a fully managed container registry that provides high-performance hosting, enabling you to deploy application images and artifacts anywhere reliably.
- It eliminates the need to operate your own container repositories or worry about scaling the underlying infrastructure.

Features

- ECR supports Docker Registry HTTP API V2 allowing you to use Docker CLI commands or your preferred Docker tools in maintaining your existing development workflow.
- ECR stores both the containers you create and any container software you buy through AWS Marketplace.
- ECR stores your container images in Amazon S3.
- ECR supports the ability to define and organize repositories in your registry using namespaces.
- You can transfer your container images to and from Amazon ECR via HTTPS.

Components

- **Registry**
  - A registry is provided to each AWS account. You have a Private Registry (default) for internal images and a Public Registry for sharing images globally.
  - The URL for your default registry is https://*aws_account_id*.dkr.ecr.*region*.amazonaws.com.
  - You must authenticate via the AWS CLI (aws ecr get-login-password) to push or pull images.
- **Authorization token**
  - Your Docker client needs to authenticate to ECR registries as an AWS user before it can push and pull images. The AWS CLI *get-login* command provides you with authentication credentials to pass to Docker.
- **Repository**
  - An object that contains your Docker images, Open Container Initiative (OCI) images, and OCI compatible artifacts.
  - ECR uses resource-based permissions to let you specify who has access to a repository and what actions they can perform on it.
  - You can configure a repository to be “immutable,” which prevents image tags from being overwritten. This ensures that a specific tag (e.g., v1.0) always points to the exact same image digest, preventing accidental changes to deployed code.
  - **ECR lifecycle policies** enable you to specify the lifecycle management of images in a repository.
- **Repository policy**
  - You can control access to your repositories and the images within them with repository policies.
- **Image**
  - Supports **Multi-architecture images**, allowing you to store variants for different CPU architectures (like x86-64 and ARM/Graviton) under a single image manifest list.
  - You can push and pull Docker images to your repositories. You can use these images locally on your development system, or you can use them in ECS task definitions.
  - You can replicate images in your private repositories across AWS regions.

Amazon Elastic Container Registry Security

- By default, IAM users don’t have permission to create or modify Amazon ECR resources or perform tasks using the Amazon ECR API.
- Use IAM policies to grant or deny permission to use ECR resources and operations.
- ECR partially supports resource-level permissions.
- ECR supports the use of customer master keys (CMK) managed by AWS Key Management Service (KMS) to encrypt container images stored in your ECR repositories.
- **VPC Endpoints (PrivateLink):** You can configure interface VPC endpoints to allow your EC2 instances, ECS tasks, or EKS pods to pull images from ECR without traversing the public internet. This ensures that your traffic stays entirely within the AWS network.
- **Image Scanning:**
  - Basic Scanning: Uses the Common Vulnerabilities and Exposures (CVEs) database from the open-source Clair project.
  - Enhanced Scanning: Integrates with **Amazon Inspector** to provide continuous, automated scanning for both operating system and programming language package vulnerabilities.
- **Image Signing:**
  - ECR integrates with **AWS Signer** (using the open-source Notation client) to cryptographically sign your container images.
  - You can then configure your orchestrators (such as EKS) to verify the image’s signature before deployment, ensuring it comes from a trusted source and hasn’t been tampered with.

From <https://tutorialsdojo.com/amazon-elastic-container-registry-amazon-ecr/>